AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Functions and arguments

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Define the procedure. Bind the arguments. Locate the output.

Distinguish function creation, calls, positional binding and a short return-value preview.

Keep one habit: Printing function → return preview → reversed arithmetic → six call cases → selected returned-difference trace.

MAIN LESSON · SECTION 1

What happens at definition time, and what happens at a call?

> Follow arguments into parameters in their stated order.

Your learning goals

I can distinguish a **function definition** from a **function call**.

I can match positional **argument**s to **parameter**s in order.

I can reuse one **function** and test that it uses the supplied values.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Distinguish a **function definition** from a **function call**
- Match positional **argument**s to **parameter**s in order
- Reuse one **function** and test that it uses the supplied values
- A script sets capacity = 12 and used = 4. What does print(capacity - used) display?

> **My starting explanation**

Your end goal

Trace **function**s and **argument**s and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Reuse a written remaining-space procedure

> A function name does not prove its body uses the arguments correctly.

Read the capacity-minus-used task

A fictional supply cupboard records capacity and the number of occupied spaces. The remaining-space calculation is capacity minus used.

Instead of copying the same instructions for every cupboard, define one **function** and **call** it with different values. A **function** makes a procedure reusable; its name alone does not guarantee that its instructions are correct.

State the input assumptions and return preview

The main remaining-space example assumes nonnegative integer counts with used<=capacity; it does not validate them. Its **function** prints a result. A short return-value preview below prepares you to read the selected transfer script: return hands a value back to the caller, whose print displays it. Week 9 will develop returned values and local state further.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Match definition, body, call and displayed result

> The return preview is a different function from the printing example.

Complete printing function and two calls

```
def show_remaining(capacity, used):
    remaining = capacity - used
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

Trace arguments to parameters

Arguments matched in order

CALL	PARAMETER VALUES	PRINTED OUTPUT
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

Each call runs the same body with the values supplied for that call.

Read definition and indentation semantics

def introduces the **definition**. The **function** name is followed by **parameter** names in parentheses and a colon. Four-space indentation marks the body. Executing the **definition** creates the **function** but does not run its body.

The two unindented **call**s then run it, producing 8 and 3 on separate lines. A blank line helps readability; indentation determines the block.

Preview return versus caller print

```
def subtract_demo(left, right):
    return left - right

value = subtract_demo(10, 4)
print(value)
```

Event	Printed output?	Value flow
Execute def	None	Function object created; body not called
show_remaining(12,4)	8 from inside the body	Arguments bind capacity/used; calculation printed

Event	Printed output?	Value flow
subtract_demo(10,4)	None from that function	return gives 6 to caller
print(value) after preview call	6	Caller displays previously returned value

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare a definition-only file with calls using new arguments

> A function should use the supplied values rather than its first example's constants.

Predict the supported change

- Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

Keep the procedure fixed. Explain what changed in the inputs and result.

Predict the pictured variant

- Change: Remove both **call**s and leave the **definition** only.

▼ Check after predicting

Prediction: No **output** is printed.

Reason: The body runs only when the **function** is **call**ed.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace the same function with reversed arguments

> Position changes the binding and therefore ordered subtraction.

Read complete pictured calls and scope

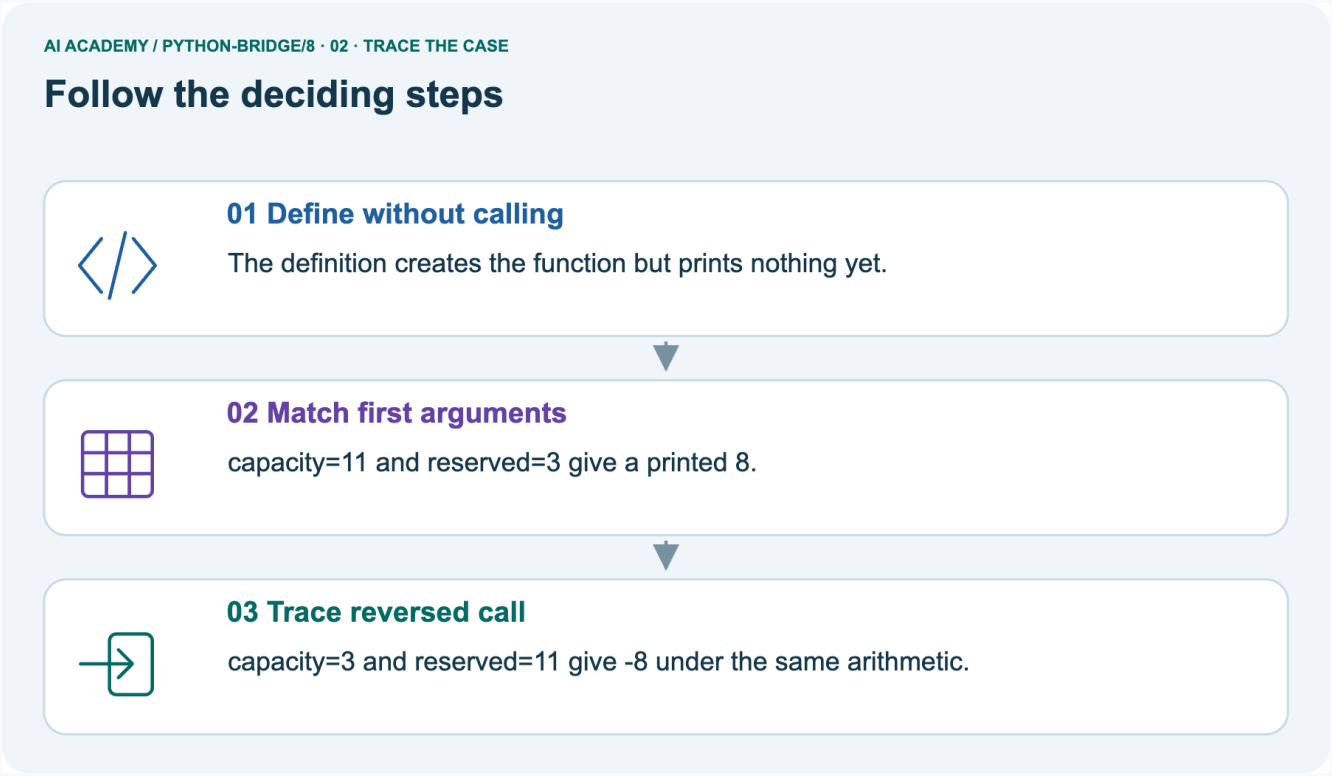
Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

Python 3: def show_free(capacity,reserved): print(capacity-reserved). **Call**s: show_free(11,3); show_free(3,11).

Separate the contracts. Main show_remaining assumes used<=capacity, so reversed (4,12) is outside its valid remaining-space domain even though arithmetic prints -8. The pictured show_free pair deliberately demonstrates subtraction order: 11-3=8 and 3-11=-8. A negative arithmetic result is not a valid free-space count under the main contract. Removing calls leaves definitions but no printed output.

The selected transfer uses a returned difference and an outer print, as previewed above.

Trace definition and two parameter bindings



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Define without calling	The definition creates the function but prints nothing yet.
2. Match first arguments	capacity=11 and reserved=3 give a printed 8.
3. Trace reversed call	capacity=3 and reserved=11 give -8 under the same arithmetic.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

Complete two-call demonstration

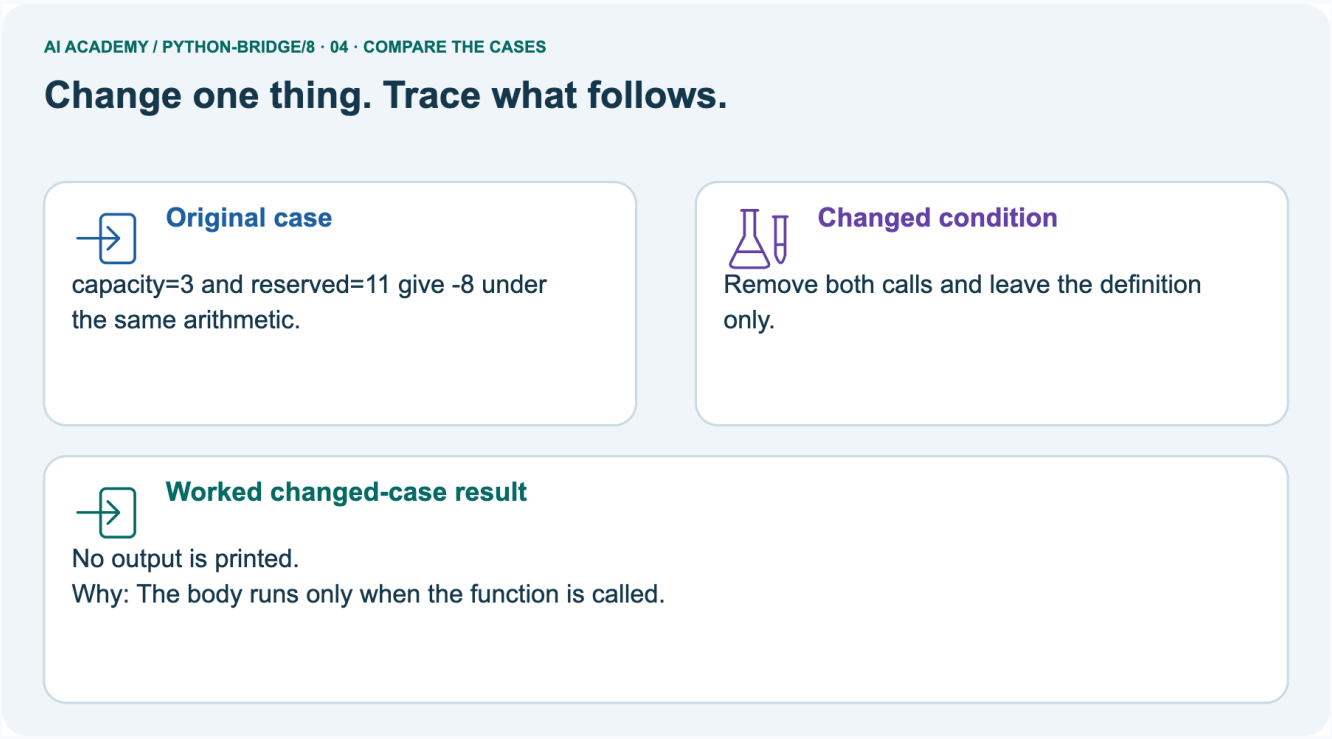
```
def show_free(capacity, reserved):
    print(capacity - reserved)

show_free(11, 3)
show_free(3, 11)
```

Predict removing both calls

- Change one thing: Remove both **call**s and leave the **definition** only. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

State arithmetic and capacity-domain limits

- The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

- Read the labelled picture: Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

▼ Reference · Main printing-function events

Execution event	What happens
Execute the definition	Create the function; no output yet

Execution event	What happens
Call with 12 and 4	Match parameters; subtract; print 8
Call with 10 and 7	Match new values; subtract; print 3
After the second call	Continue with the next statement

In show_remaining(12, 4), 12 and 4 are **argument**s; capacity and used are **parameter**s.

Positional **argument**s match from left to right. show_remaining(4, 12) produces -8, showing why order matters; it also violates our written **input** contract. Use a fresh file with the **definition** before its **call**s.

> **Predict the output of show_remaining(9, 9) followed by show_remaining(9, 0). Name the parameter receiving each argument. What would be printed if both calls were removed?**

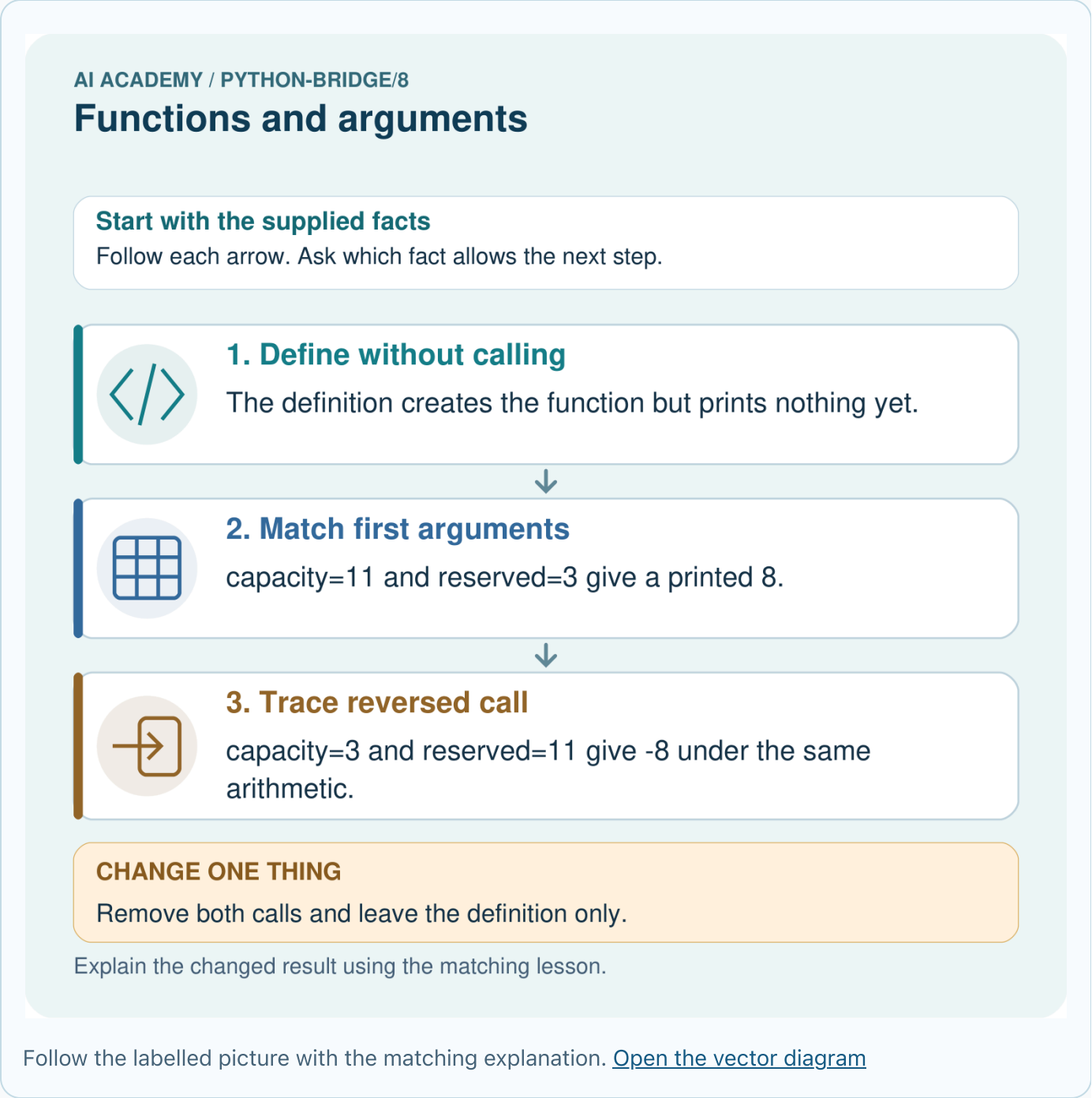
▼ **Earlier wording, alternatives and picture transcripts**

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Functions and arguments

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find constants that ignore the function’s arguments

> A first example can pass by accident while reuse fails.

Read the complete faulty body

```
def show_remaining(capacity, used):
    remaining = 12 - 4
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

Compare actual repeated 8 with expected new inputs

This faulty **function** prints 8 twice. Its fixed numbers ignore the **argument**s. Replace 12 - 4 with capacity - used, then rerun both **call**s.

Testing only the first **call** would miss the bug because its expected answer happens to be 8.

Distinguish a missing argument from no call

For this two-**parameter definition**, show_remaining(12) raises TypeError because an **argument** is missing. Correct the **call** to supply both values.

Naming a **function** without **call**ing it does not execute the body: the parentheses in show_remaining(12, 4) are part of the **call**.

Repair and choose a regression call

- The faulty body calculates 12 - 4 regardless of the **call**. State the **output** and expected **output** for show_remaining(11, 2). Repair the expression and name an earlier **call** to rerun.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise calls and parameter binding with less help

> Predict both the output and whether the body runs.

Explain the procedure

- In def show_remaining(capacity, used), identify the **function** name and **parameter**s. In show_remaining(15, 6), identify the **argument**s and their matching **parameter**s.

Predict the output of show_remaining(9, 9) followed by show_remaining(9, 0). Name the parameter receiving each argument. What would be printed if both calls were removed?

> **Explain your prediction to a partner before comparing results.**

Complete W2 call sequence

Use workbook W2 for the connected example “**Function**s and **argument**s”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Audit definition-runs-body reasoning

- Use the worked case for **Function**s and **argument**s. Proposed explanation: “Writing def runs the body immediately”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured argument reversal

- Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

My explanation using a pictured step and the given facts

Reserve selected transfer work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Follow the execution order

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check six complete definition and call cases

> Record argument bindings, whether the body runs and where output is produced.

All supplied call and failure records

Complete case	Expected event/result
A · Correct show_remaining(8,3)	Bind capacity 8/used 3; print 5

Complete case	Expected event/result
B · Correct show_remaining(8,8)	Print 0; a legitimate full cupboard
C · Correct show_remaining(8,0)	Print 8; zero used spaces
D · File has only correct def block, no call	No printed output; body never runs
E · Correct two-parameter definition, call show_remaining(8)	TypeError before body runs; required argument absent
F · Faulty fixed 12-4 body called with (11,2)	Print 8 instead of expected 9; supplied values ignored

Complete W2 three-call script

```
def show_remaining(capacity, used):
    remaining = capacity - used
    print(remaining)

show_remaining(8, 3)
show_remaining(8, 8)
show_remaining(8, 0)
```

Predict, run or trace and repair the ignored arguments

1. Use the complete W2 script to predict A/B/C, then run or trace and preserve three output lines. Create D by removing calls only, and E by using the correct definition plus one missing-argument call in a separate fresh file. F uses the supplied faulty body: compare its output to the written capacity-minus-used requirement, repair to parameter names and rerun a different call.

   These main cases use valid nonnegative counts, apart from the explicit call-shape error; arbitrary value validation remains unimplemented. Reversed arithmetic in the picture illustrates order, not a valid capacity decision. Do not call a displayed number a returned value unless the function has a return statement. Keep selected difference(11,4)/difference(4,11) for the individual attempt.

Keep the evidence record

Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Reference · Original correct and faulty definitions

```
def show_remaining(capacity, used):
    remaining = capacity - used
    print(remaining)

show_remaining(12, 4)
show_remaining(10, 7)
```

```
def show_remaining(capacity, used):
    remaining = 12 - 4
    print(remaining)
```

```
show_remaining(12, 4)
show_remaining(10, 7)
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected returned differences

> Separate the returned value, outer display and definition-only behavior.

Selected complete task and scope

Predict both returned and printed values, showing which **argument** supplies first and second. What is displayed if the program contains only the **function definition** and no **call**s?

Why does swapping these **argument**s change the result?

```
def difference(first, second):
    return first - second
print(difference(11, 4))
print(difference(4, 11))
```

Use the selected function's own definition: it returns an arithmetic difference, and the surrounding print displays that returned value. Map arguments to first/second in order. The printing-capacity example's nonnegative remaining-space contract is separate; here subtraction order may legitimately give a negative numeric result.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier printing-function transfer

Each sealed box contains 6 counters. Define show_stock(boxes, loose) to print the total counters using its two **parameter**s. Assume nonnegative integer **argument**s. **Call** it with (3, 2), (0, 2) and (3, 0).

Predict each **output** and explain why show_stock(2, 3) is a different case.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Match positional arguments to parameters

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep parameter binding and execution order in the portfolio

> A reusable procedure needs tests with more than its first example's values.

This week's contribution

Add a labelled **function**s and **argument**s **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **function definition** from a **function call**.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain definition, call, binding and output source

> A body uses current arguments when called, rather than copying a first answer.

Explain without notes

- Close your notes. Explain this goal in your own words: Distinguish a **function definition** from a **function call**

> **My deciding step, evidence and one remaining question**

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Function	A named block of reusable instructions that runs when called
Definition	Code that creates a function and specifies its body
Call	An instruction that runs a function with supplied arguments
Parameter	A name in a function definition that receives an argument
Argument	A value supplied when calling a function

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- In def show_remaining(capacity, used), identify the **function** name and **parameter**s. In show_remaining(15, 6), identify the **argument**s and their matching **parameter**s.

- The faulty body calculates 12 - 4 regardless of the **call**. State the **output** and expected **output** for show_remaining(11, 2). Repair the expression and name an earlier **call** to rerun.

- The correct subtraction **function** prints -1 for show_remaining(4, 5). Does successful execution make this a valid cupboard record? Explain what the **function** assumes and what needs checking.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- For each **call**, record the supplied **argument**s, expected printed **output** and observed **output**. Include different values that produce different answers, plus a valid zero-result case.

  Call order controls **output** order; it does not change the subtraction rule. These **function**s use supplied data and written instructions, not a pattern learned from examples.

- The correct subtraction **function** prints -1 for show_remaining(4, 5). Does successful execution make this a valid cupboard record? Explain what the **function** assumes and what needs checking.

> **A learner tests only show_remaining(12, 4) and says the function works for every cupboard. Give a second valid call that exposes the fixed-number bug, state its expected output and explain what passing two tests still cannot prove.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

Arguments matched in order

CALL	PARAMETER VALUES	PRINTED OUTPUT
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

Each call runs the same body with the values supplied for that call.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Distinguish a function definition from a function call	Explain the deciding step, show a trace or test, and state one limit.
Match positional arguments to parameters in order	Explain the deciding step, show a trace or test, and state one limit.
Reuse one function and test that it uses the supplied values	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.
- Which labelled step matches this detail? capacity=3 and reserved=11 give -8 under the same arithmetic.
- 1. Match first **argument**s
- 2. Trace reversed **call**
- 3. Define without **call**ing
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Trace reversed **call** capacity=3 and reserved=11 give -8 under the same arithmetic. Positional **argument**s match **parameter** names in order.

A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

Changed-case question and hint

- Changed-case practice: Remove both **call**s and leave the **definition** only. Which conclusion is supported by the supplied facts?
- 1. This one example guarantees correct results for every future **input**.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. No **output** is printed.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: No **output** is printed. The body runs only when the **function** is **call**ed.

Repair and keep the record

Review Lists and indexing (python-bridge/7). Goal: Distinguish a list index from the value stored at that position. Short practice: Label the three index positions.

Predict all original **output**s, then name the first **output** and stopping point after using index 3.. Return to this week and explain what you changed.

Review Branching and boundary tests (python-bridge/5). Goal: Trace the selected branch in an if elif else chain. Short practice: Trace -2, 4 and 5 through the branches.

Explain why the negative check comes first and what label -2 gets if it is removed.. Return to this week and explain what you changed.

Review Values variables and types (python-bridge/2). Goal: Distinguish integer, floating-point, text and Boolean values. Short practice: Predict both print **output**s and explain why they differ.

Does count+tag work, and what happens to each valid **output** after count becomes ten?. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

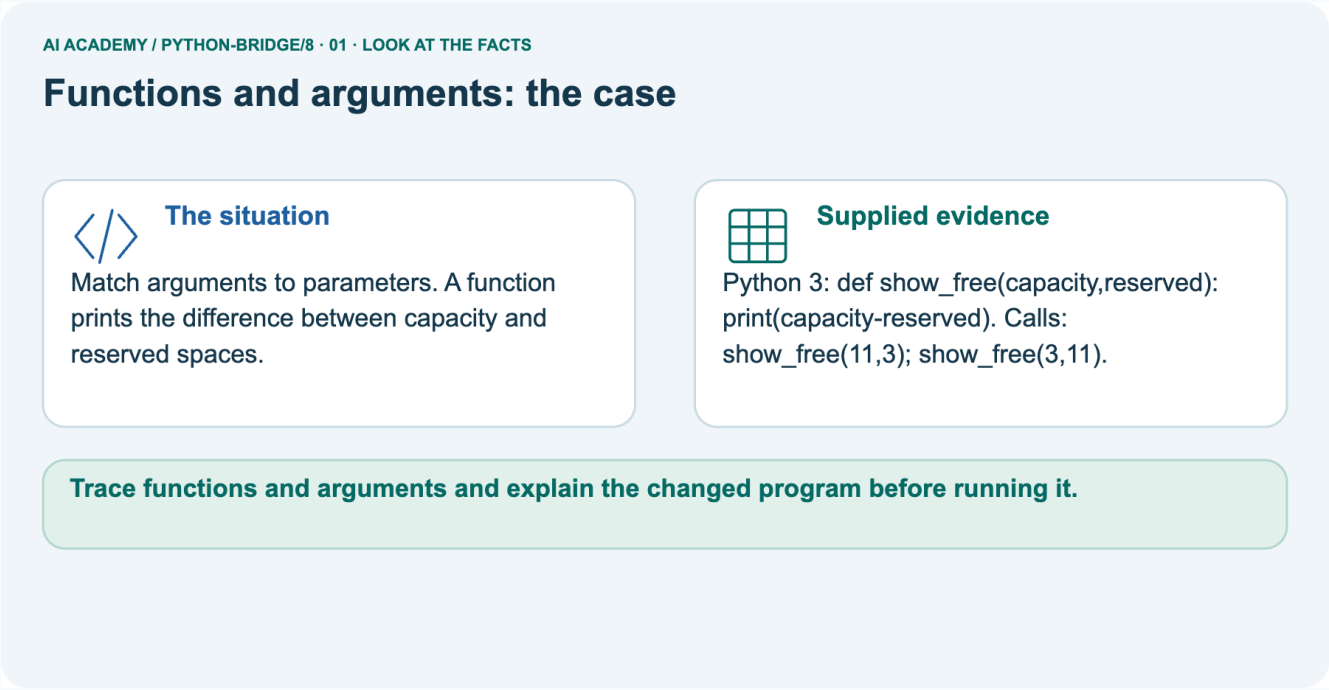
Keep your completed work

▼ Reference · The six-picture journey

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Functions and arguments: the case



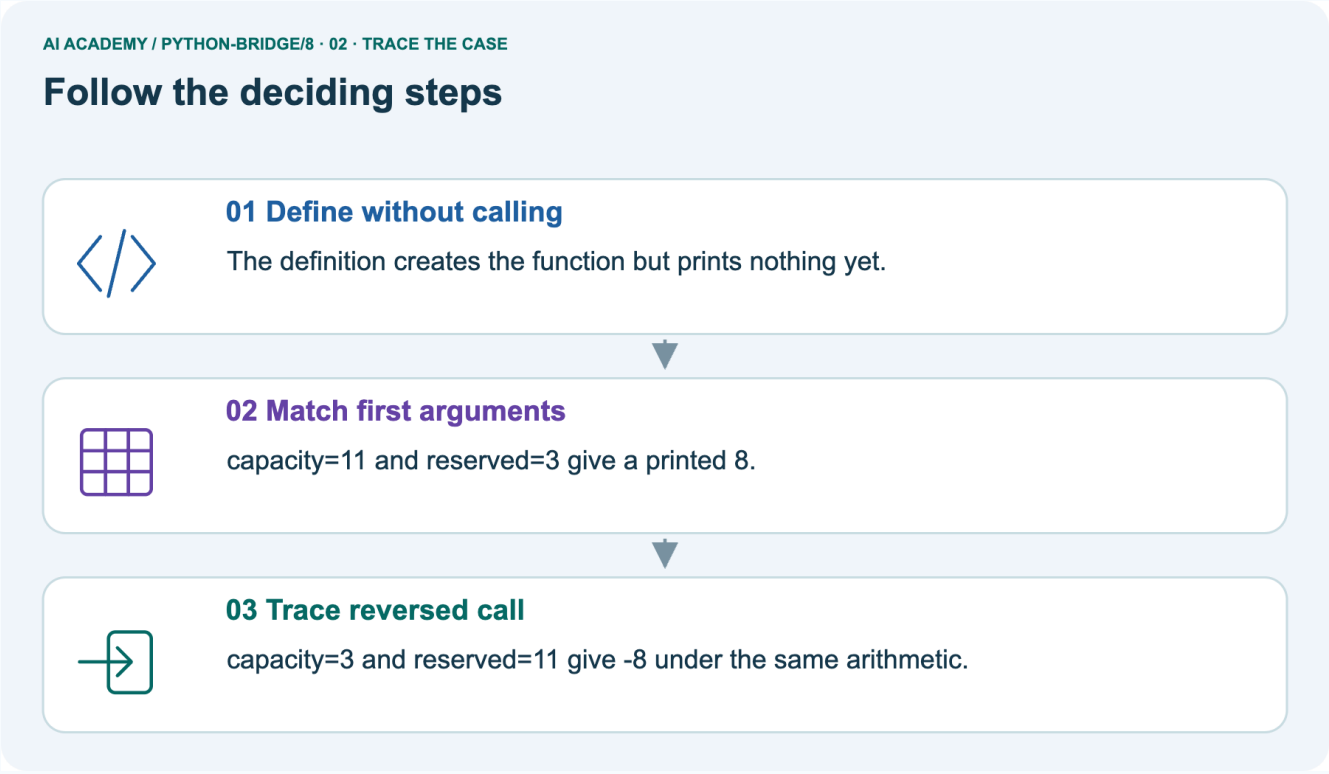
Follow the labelled picture with the matching explanation. Open the vector diagram

Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces. Supplied facts: Python 3: def show_free(capacity,reserved): print(capacity-reserved).

Calls: show_free(11,3); show_free(3,11).

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

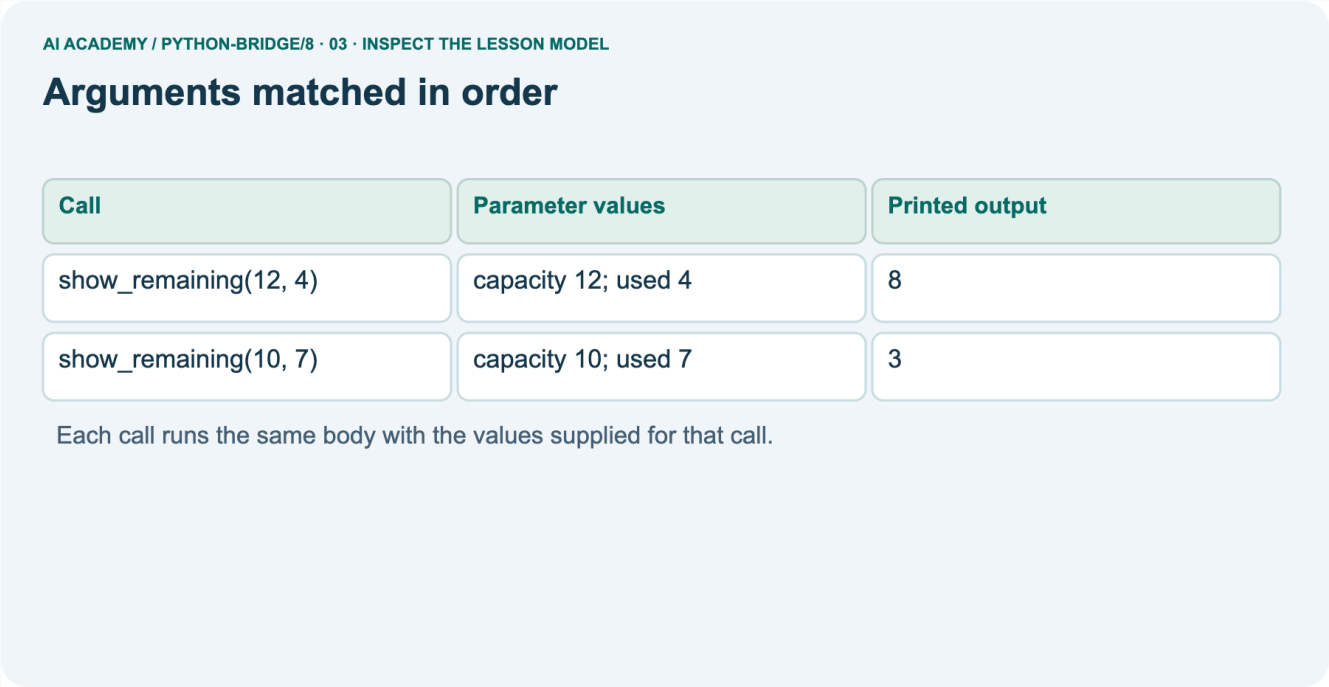


Follow the labelled picture with the matching explanation. Open the vector diagram

Define without **call**ing: The **definition** creates the **function** but prints nothing yet. → Match first **argument**s: capacity=11 and reserved=3 give a printed 8. → Trace reversed **call**: capacity=3 and reserved=11 give -8 under the same arithmetic.

Point to each numbered stage and explain which recorded **input** it uses.

03. Arguments matched in order

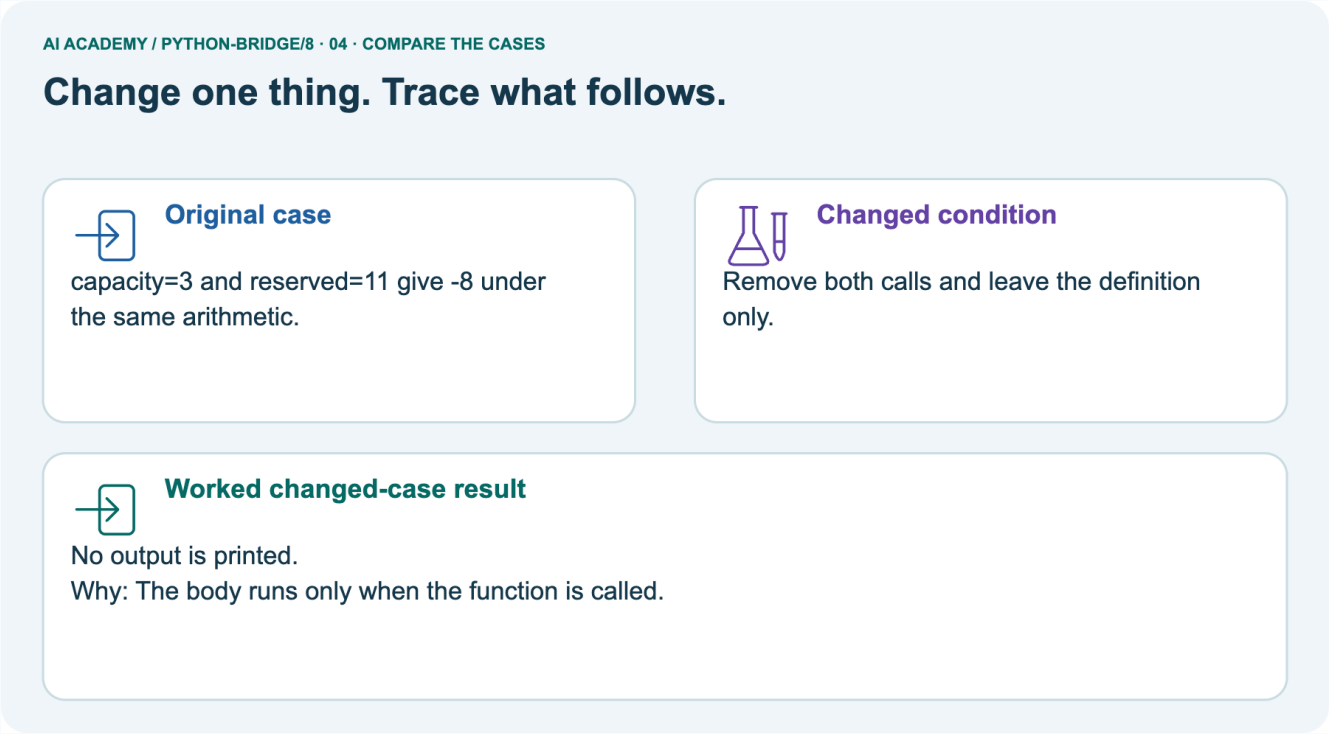


Follow the labelled picture with the matching explanation. Open the vector diagram

Arguments matched in order. show_remaining(12, 4) | capacity 12; used 4 | 8. show_remaining(10, 7) | capacity 10; used 7 | 3

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



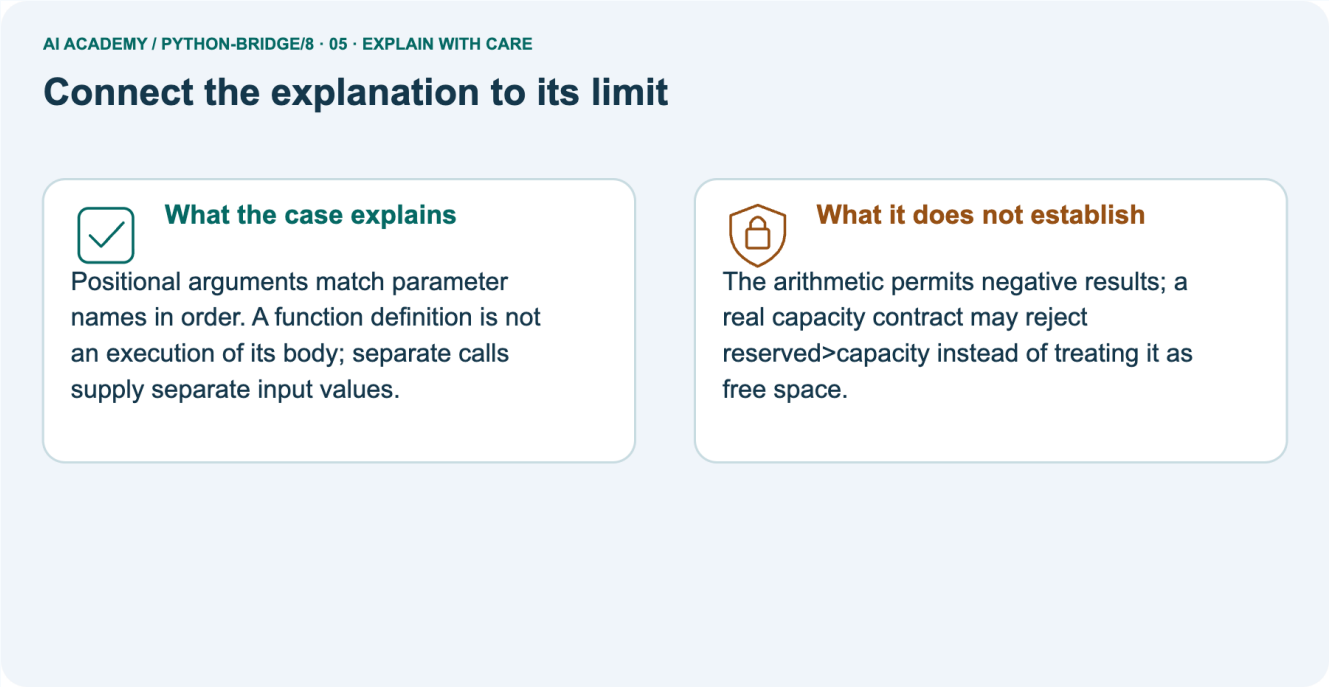
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: capacity=3 and reserved=11 give -8 under the same arithmetic. Changed condition: Remove both **call**s and leave the **definition** only. Expected result: No **output** is printed.

Reason: The body runs only when the **function** is **call**ed.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



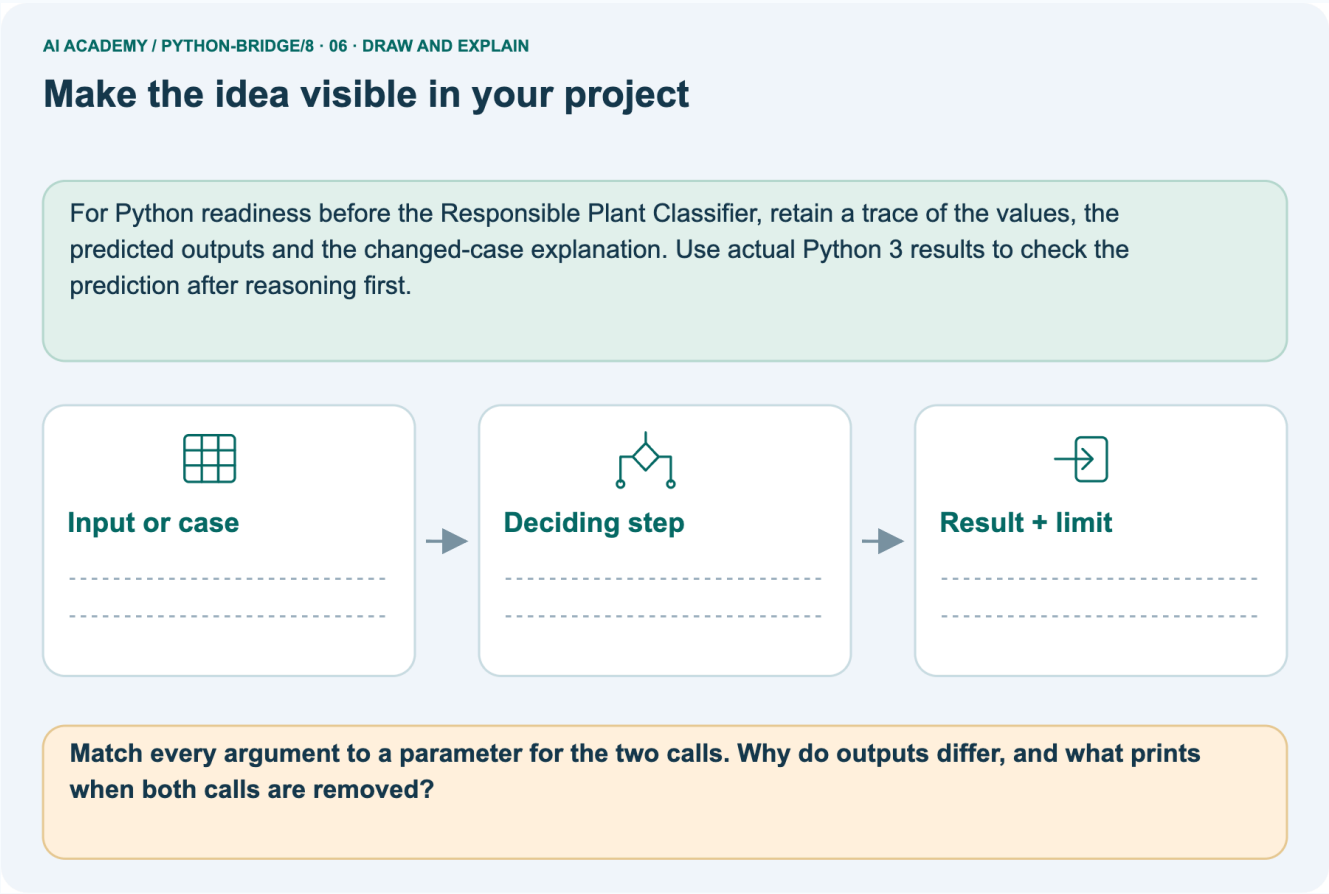
Follow the labelled picture with the matching explanation. Open the vector diagram

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

Limit: The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Functions and arguments: the case

02. Follow the deciding steps

03. Arguments matched in order

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.